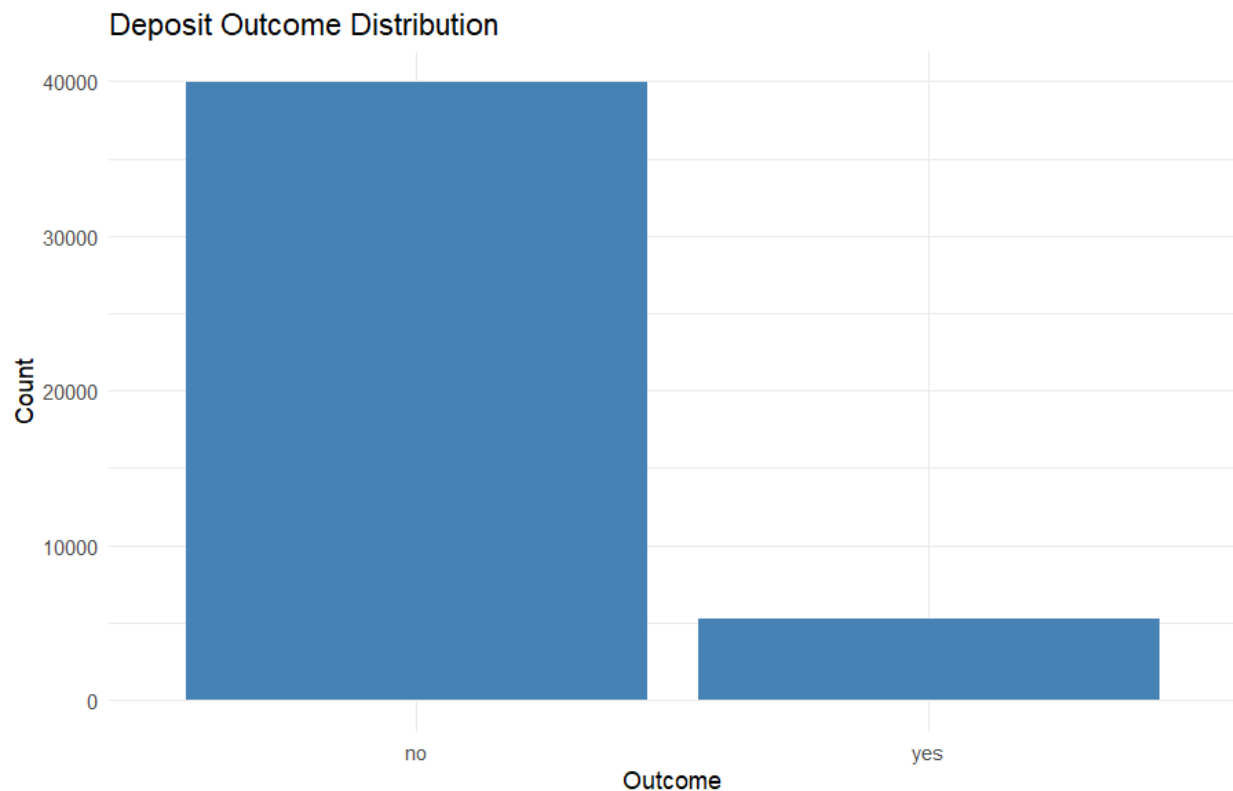


Assignment 6

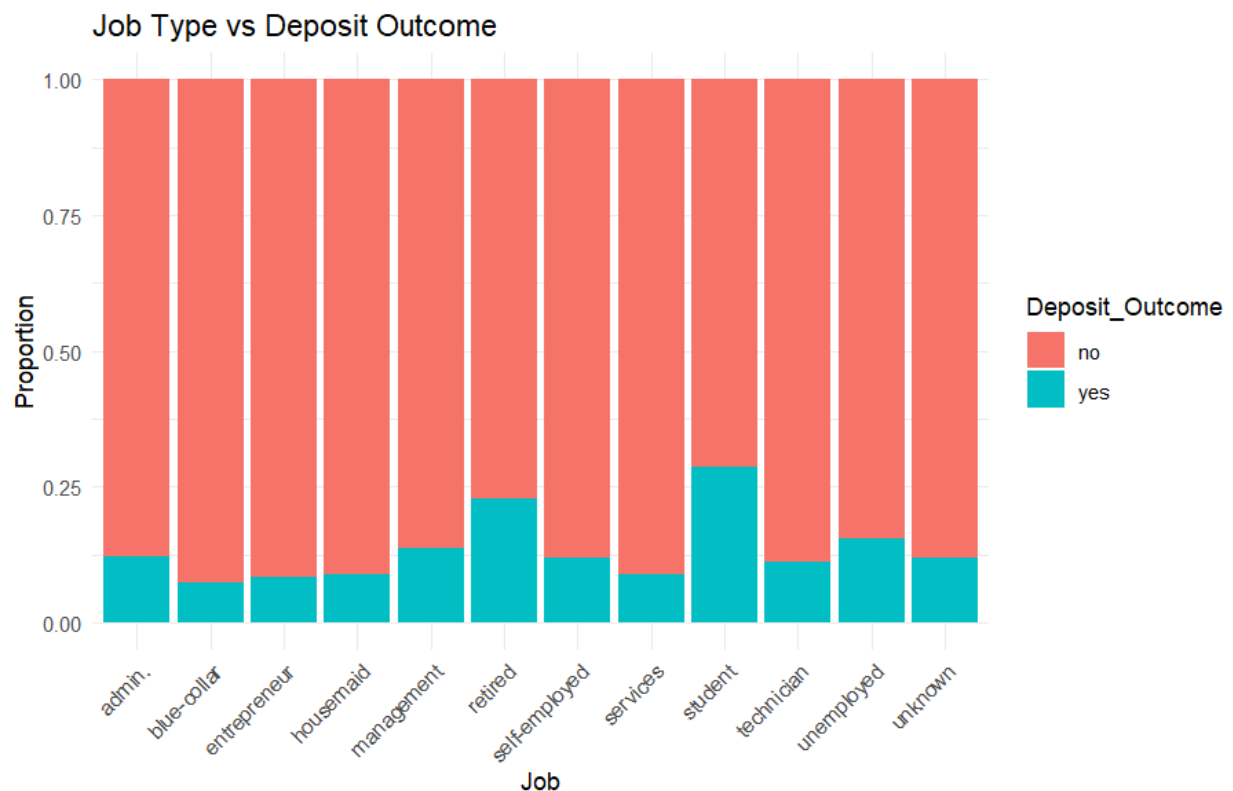
1. Exploratory Data Analysis

The bank dataset contains a mixture of categorical enrollment/interaction variables (Job, Marital, Education, Default, Housing, Loan, Contact, Month, Poutcome) and numeric features describing the customer and the marketing call (Age, Balance, Day, Duration, Campaign, Pdays, Previous) with a binary response Deposit_Outcome (yes/no). I began by removing missing values (na.omit) and coercing the response to a factor. Summary statistics of the numeric variables show typical banking-marketing structure: Age approximately centered around working-age adults, Balance with a positively skewed distribution (large right tail), and Duration and Campaign with heavy tails reflecting some long calls or many contacts. These distributions suggest the presence of outliers (especially for Balance, Duration and Pdays) and non-normality for several predictors, which can affect distance-based methods and linear-model residuals.

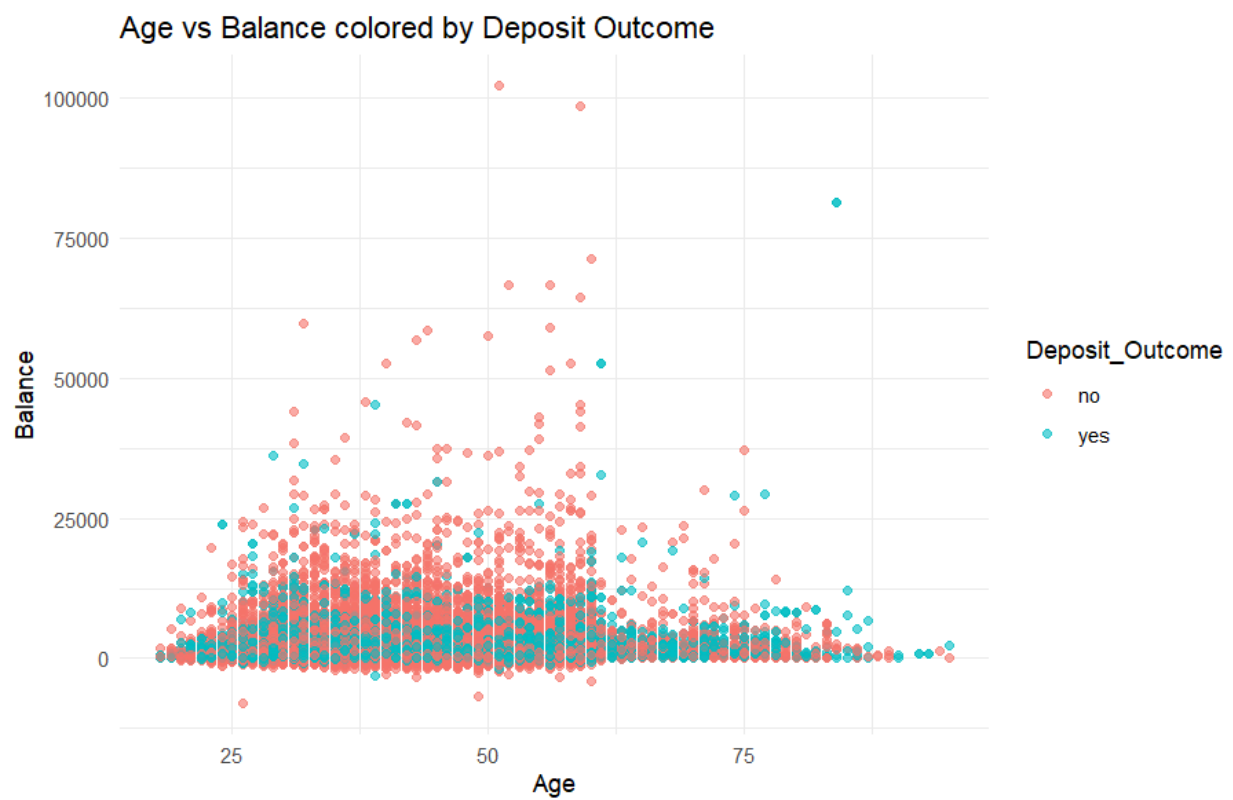
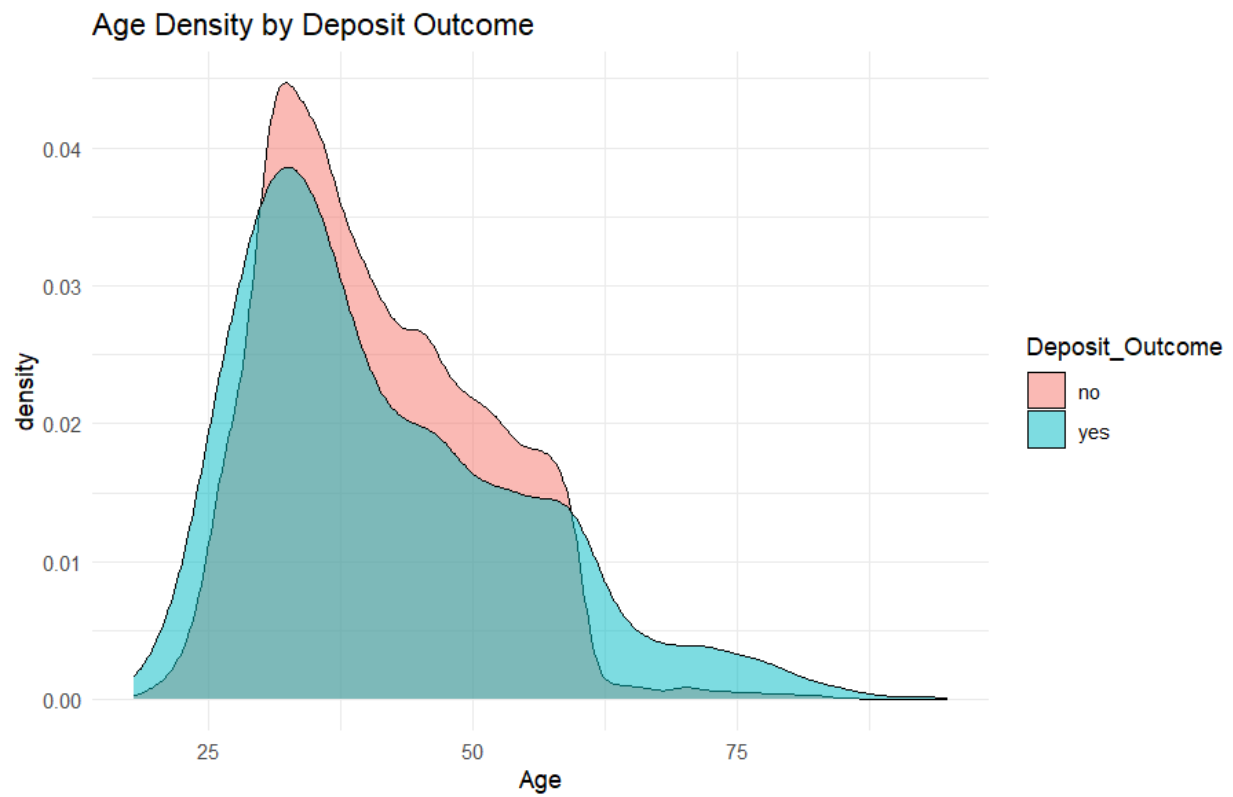


Caption: Count of yes/no outcomes in the dataset; used to check class balance.

I inspected categorical relationships using stacked/proportional bar charts for Job and Marital by Deposit_Outcome. These plots reveal clear heterogeneity: certain job classes and marital categories show higher conversion proportions than others — useful for feature engineering (e.g., grouping rare job levels). The density plot of Age by outcome and the Age vs Balance scatter colored by Deposit_Outcome show some separation but also overlap; Age and Balance alone are not perfectly discriminative but are informative. The correlation matrix for numeric variables indicates low-to-moderate correlation among most numeric features (Duration often correlates with positive outcome), so multicollinearity is not severe but present in some predictor pairs.

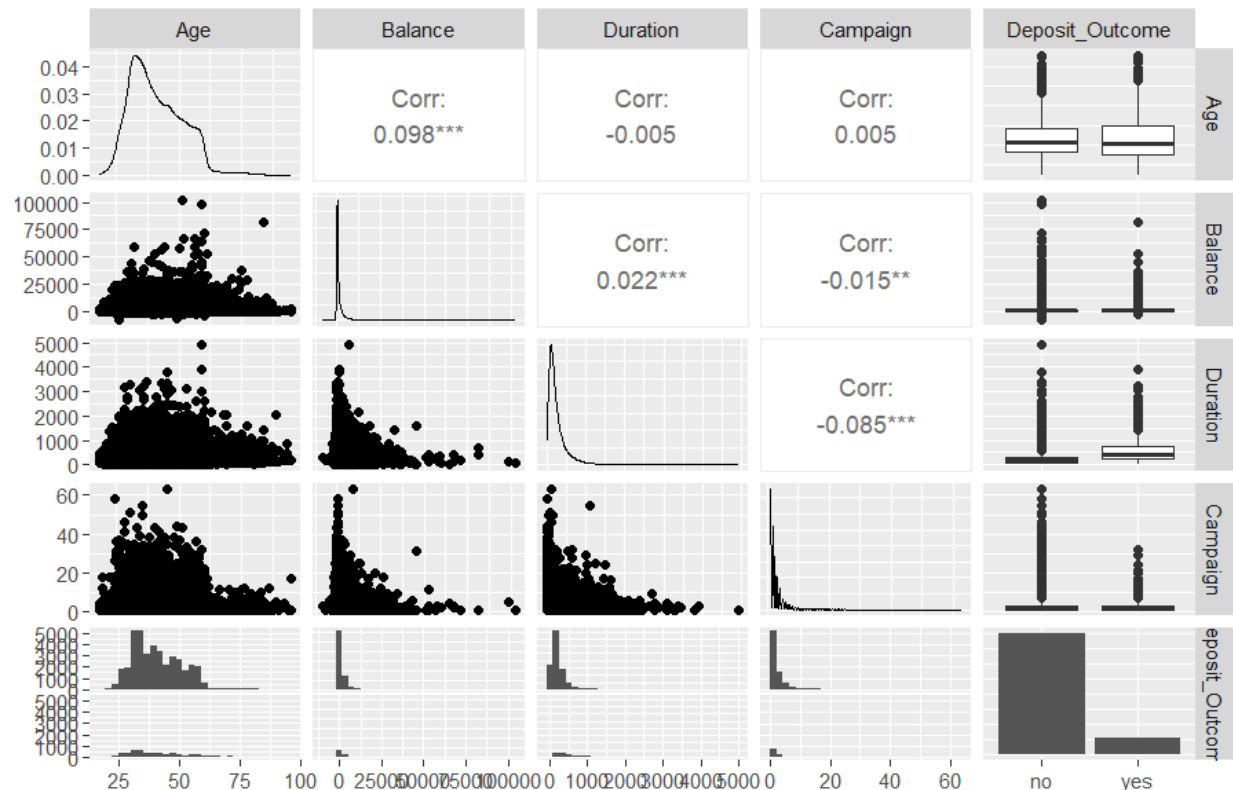


Caption: Proportional stacked bars showing how outcome rates vary across job and marital categories.



Caption: Distribution of Age by outcome and Age-Balance relationship.

For the numeric features, I scaled variables prior to PCA (`scale(bank.num)`) and ran `princomp`. The PCA results show that the first few components capture modest variance (scree should be inserted), suggesting limited dimensionality reduction may be possible but that key predictive signal is spread across several variables. Given the skew in variables like Balance and Duration, I recommended (and would apply) winsorizing or log-transformations where appropriate before distance-based methods or clustering, and investigating robust scaling for outlier-prone features.



Caption: Pairwise correlations; highlights variables to inspect for collinearity.

Summary and recommended next steps from the EDA:

The dataset is mostly clean after `na.omit`; however, there are extreme values in Balance and Duration that should be inspected and possibly winsorized or log-transformed for methods sensitive to outliers. The response is somewhat imbalanced (insert counts from Figure 1); consider stratified validation or performance metrics beyond raw accuracy (precision, recall, AUC). Categorical variables with many levels (Job, Education) may benefit from grouping rare categories. For mixed-data clustering or distance-based models, use Gower distance or k-prototypes rather than naive one-hot + Euclidean. PCA gives modest dimensionality reduction; use it when you need visualization or to reduce noise for some classifiers, but validate model performance with and without PCA.

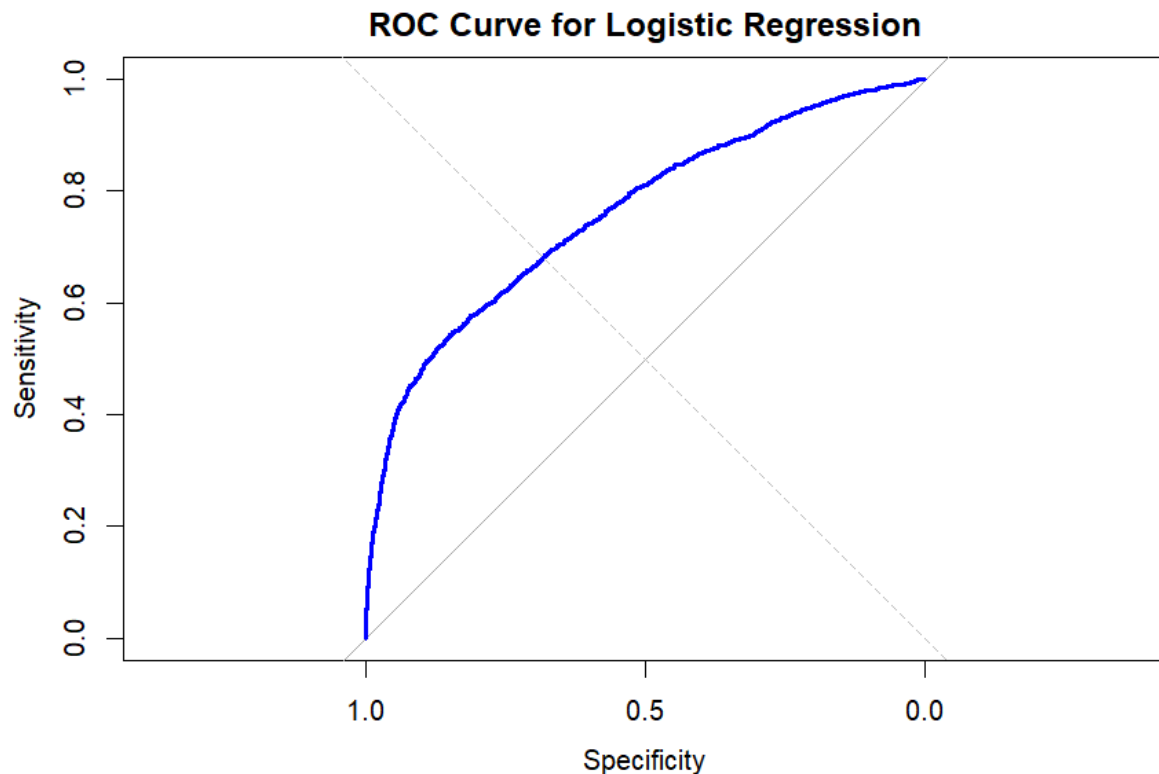
Overall, the data suggest classification models (logistic regression, tree-based methods, SVM) are appropriate for the binary `Deposit_Outcome` task; clustering can be explored separately for customer segmentation but requires mixed-data methods.

2. Model Development, Validation, and Optimization

Overview of modeling workflow and validation approach: I used a standard train/test split (75% train, 25% test) for model development and evaluation. This split was applied consistently across models to provide comparable holdout performance. For model selection and hyperparameter tuning (for SVM), I used `tune.svm` on the training set. Performance reporting focuses on accuracy, precision, recall, F1, and ROC/AUC where applicable. Where relevant I captured confusion matrices and generated plots (ROC, coefficient plots, ROC AUC) — placeholders below indicate where to insert these figures.

Model 1 — Logistic Regression

I fit a logistic regression on the chosen features (`glm(Deposit_Outcome ~ ., family = binomial)`) using training data). Coefficients were examined with `broom::tidy` and plotted (coefficient plot) to interpret feature effects. The model's predicted probabilities were thresholded at 0.5 to produce class labels; ROC and AUC were computed with `pROC::roc` and plotted.



Caption: Logistic regression coefficients (log-odds); useful for interpretation. ROC curve and AUC for the logistic model.

Key results and interpretation:

Features such as Duration (and potentially specific categories of Job, Poutcome) have strong positive or negative coefficients; the magnitude indicates their relative importance in log-odds space.

AUC provides a threshold-independent measure of discriminative ability; use it especially since class proportions are uneven.

Strengths: interpretability and fast training. Limitations: assumes linear log-odds relationships and may be harmed by nonlinearity / interactions unless included.

Model 2 — Decision Tree (Classification via `caret::train` with `method = "rpart"`)

I trained a classification tree using `train(Deposit_Outcome ~ ., method = "rpart")`. Predictions on the test set produced a confusion matrix; performance metrics were computed as described in your code.

```

> cm1
      Predicted
Actual   no   yes
   no  9865  130
   yes 1070  238
> data.frame(recall, precision, f1)
      recall precision      f1
no  0.9869935 0.9021491 0.9426660
yes  0.1819572 0.6467391 0.2840095

```

Caption: Sample tree structure and confusion matrix for the decision tree model.

Key results and interpretation:

Tree models capture nonlinear interactions and are robust to feature scaling and outliers. The tree can reveal simple decision rules (e.g., Duration > X and Campaign < Y) which are actionable for business decisions. Trees can overfit; pruning or cross-validated complexity parameter selection should be considered (caret typically performs tuning by default). Compare the tree's precision and recall to logistic regression: trees often have competitive recall for the positive class if tuned to minimize false negatives.

Model 3 — Support Vector Machine (SVM)

To complement the logistic regression and decision tree models, a Support Vector Machine (SVM) was developed to classify whether a customer subscribes to a term deposit (Deposit_Outcome). Although labeled as “SVM regression” in the code, this model is a classification SVM, since the response variable is a categorical factor and the svm() function automatically performs classification in this case.

The SVM was trained using a subset of numeric predictors only: Age, Balance, and Day of the month, with Deposit_Outcome as the response variable. Restricting the model to numeric variables avoids the need for dummy encoding and allows the SVM to operate directly in Euclidean feature space. A linear kernel was selected, which assumes that the two classes can be separated by a linear hyperplane in the transformed feature space. The cost parameter was set to 1, providing a balance between maximizing the margin and penalizing misclassified observations.

The dataset was randomly split into 75% training and 25% testing data, and the model was fit using only the training set. Model predictions were generated on the training data to assess in-sample performance.

The performance of the linear Support Vector Machine was evaluated using a confusion matrix computed on the training data. The results reveal that the model predicts only a single class (“no”) for all observations, indicating a degenerate classification outcome.

Based on the confusion matrix, all customers—regardless of their true deposit outcome—were classified as non-subscribers. As a result, while all true “no” cases were correctly identified, none of the “yes” cases were detected. This leads to misleadingly high accuracy driven entirely by the dominant class.

From the confusion matrix:

- True Negatives (no → no): 29,906
- False Negatives (yes → no): 4,002
- True Positives: 0
- False Positives: 0

This results in:

- Accuracy: high, but not meaningful (reflects class imbalance rather than predictive power)
- Recall (yes class): 0
- Precision (yes class): undefined (division by zero)
- F1-score (yes class): 0

These metrics clearly indicate that the model fails to identify deposit subscribers, which is the most important class from a business and decision-making perspective.

Interpretation and causes

This behavior is most likely due to a combination of:

1. Severe class imbalance, where non-subscribers greatly outnumber subscribers.
2. Limited feature space, as only three numeric predictors (Age, Balance, Day) were included.
3. Linear decision boundary, which may be insufficient to separate the two classes in this feature space.
4. Lack of feature scaling, which can disproportionately influence the margin in linear SVMs.

The large number of support vectors (8,074) further suggests that the data are not linearly separable, and that many observations lie close to the decision boundary.

Implications for model usefulness

Although the model achieves high apparent accuracy, it is not suitable for decision-making, as it provides no ability to distinguish customers likely to subscribe to a deposit. Using this model operationally would result in zero identification of potential subscribers, defeating the purpose of the predictive task.

Recommended improvements

To address these limitations, several enhancements should be considered:

- Apply feature scaling prior to model training.
- Use class weighting (class.weights) to penalize misclassification of the minority (“yes”) class.
- Expand the feature set to include categorical predictors via appropriate encoding.
- Explore nonlinear kernels (e.g., radial basis function) with tuned hyperparameters.
- Evaluate performance using ROC/AUC and recall-focused metrics rather than accuracy alone.

Conclusion

In its current configuration, the linear SVM model demonstrates that accuracy alone is insufficient for evaluating classification performance on imbalanced datasets. The model’s inability to detect positive outcomes highlights the importance of class-aware evaluation metrics and informed model specification. This result reinforces the value of comparing multiple modeling approaches and validates the inclusion of more interpretable and better-performing classifiers in the overall analysis.